

Documentación del examen práctico - Unidad 1

Keyla Bedón Viteri

June 2026

1. Introduction

Se implementó un prototipo funcional del servicio RegistroDistribuido como parte del Examen Práctico de la Unidad 1. El sistema simula una empresa de logística que opera con tres nodos réplica (N1, N2, N3) capaces de recibir operaciones de registro de movimientos desde uno o varios clientes.

El prototipo demuestra los cinco mecanismos fundamentales de sistemas distribuidos:

- **Parte A** — Comunicación entre procesos mediante sockets TCP con framing.
- **Parte B** — Relojes lógicos de Lamport para orden total consistente de eventos.
- **Parte C** — Tolerancia a fallos mediante latidos (heartbeats).
- **Parte D** — Coordinación con el algoritmo Bully para elección de líder.
- **Parte E** — Seguridad básica mediante autenticación por token.

Stack tecnológico:

Java 21

Spring Boot 4.0.6

Maven

Comunicación REST sobre HTTP (TCP)

Ejecución en 3 procesos con puertos 8081, 8082 y 8083.

Repositorio: <https://github.com/BedonViteri/Aplicaciones-Distribuidas.git>

2. Estructura del Proyecto

```
registro-distribuido/
|-- src/
|   |-- main/
|   |   |-- java/com/logistica/registro_distribuido/
|   |   |   |-- controller/
|   |   |   |   '-- OperacionController.java
|   |   |   |-- model/
|   |   |   |   |-- Operacion.java
|   |   |   |   '-- SocketMessage.java
|   |   |   |-- service/
|   |   |   |   |-- AuthService.java
|   |   |   |   |-- BullyElectionService.java
|   |   |   |   |-- HeartbeatService.java
|   |   |   |   |-- LamporkClock.java
|   |   |   |   '-- NodoComunicacionService.java
|   |   |   '-- RegistroDistribuidoApplication.java
|   |   '-- resources/
|   |       '-- application.properties
|   '-- test/java/com/logistica/registro_distribuido/
|       '-- RegistroDistribuidoApplicationTests.java
```

```
| -- HELP.md  
| -- mvnw  
| -- mvnw.cmd  
' -- pom.xml
```

3. Configuración Maven — pom.xml

Archivo de configuración central del proyecto. Define dependencias y plugins de compilación. Generado desde `start.spring.io` con Lombok agregado manualmente.

Listing 1: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
    https://maven.apache.org/xsd/maven-4.0.0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  <parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>4.0.6</version>  
    <relativePath/>  
  </parent>  
  <groupId>com.logistica</groupId>  
  <artifactId>registro-distribuido</artifactId>  
  <version>0.0.1-SNAPSHOT</version>  
  <properties>  
    <java.version>21</java.version>  
  </properties>  
  <dependencies>  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-webmvc</artifactId>  
    </dependency>  
    <dependency>  
      <groupId>org.projectlombok</groupId>  
      <artifactId>lombok</artifactId>  
      <optional>true</optional>  
    </dependency>  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-webmvc-test</artifactId>  
      <scope>test</scope>  
    </dependency>  
  </dependencies>  
  <build>  
    <plugins>  
      <plugin>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-maven-plugin</artifactId>  
        <configuration>  
          <excludes>  
            <exclude>  
              <groupId>org.projectlombok</groupId>  
              <artifactId>lombok</artifactId>  
            </exclude>  
          </excludes>  
        </configuration>  
      </plugin>  
    </plugins>  
  </build>  
</project>
```

4. Clase Principal — RegistroDistribuidoApplication.java

Punto de entrada de la aplicación Spring Boot. La anotación `@SpringBootApplication` activa la configuración automática y el escaneo de componentes. Levanta el servidor Tomcat embebido en el puerto configurado.

Listing 2: RegistroDistribuidoApplication.java

```
package com.logistica.registro_distribuido;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RegistroDistribuidoApplication {

    public static void main(String[] args) {
        SpringApplication.run(RegistroDistribuidoApplication.class, args);
    }
}
```

5. Capa de Modelo (model/)

5.1. Operacion.java

Representa un evento o registro dentro del sistema distribuido. Es el objeto central que se propaga entre nodos. Las anotaciones de Lombok eliminan getters, setters y constructores manuales.

- **id**: Identificador único de la operación (UUID).
- **descripcion**: Contenido del registro. Ej: *“Movimiento de carga #4521”*.
- **timestamp**: Valor del reloj lógico de Lamport. No es tiempo real; es un contador para ordenar eventos causalmente.
- **nodoOrigen**: Qué nodo recibió primero la operación ("N1", "N2" o "N3"). Criterio de desempate ante timestamps iguales.

Listing 3: Operacion.java

```
package com.logistica.registro_distribuido.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Operacion {
    private String id;
    private String descripcion;
    private long timestamp;
    private String nodoOrigen;
}
```

5.2. SocketMessage.java

Mensaje estructurado que se transmite entre nodos por sockets TCP. Implementa `Serializable` para serialización. Usa getters y setters explícitos en lugar de Lombok.

- **type**: Tipo de mensaje: "HEARTBEAT", ".ELECTION", ".COORDINATOR", ".OPERACION".

- **payload:** Contenido del mensaje en texto.
- **lamportTimestamp:** Marca de tiempo lógica del nodo emisor al momento del envío.
- **senderId:** ID numérico del nodo que envía (1, 2 o 3).

Listing 4: SocketMessage.java

```
package com.logistica.registro_distribuido.model;

import java.io.Serializable;

public class SocketMessage implements Serializable {
    private String type;
    private String payload;
    private long lamportTimestamp;
    private int senderId;

    public SocketMessage() {}

    public SocketMessage(String type, String payload, long lamportTimestamp, int senderId) {
        this.type = type;
        this.payload = payload;
        this.lamportTimestamp = lamportTimestamp;
        this.senderId = senderId;
    }

    public String getType() { return type; }
    public void setType(String type) { this.type = type; }

    public String getPayload() { return payload; }
    public void setPayload(String payload) { this.payload = payload; }

    public long getLamportTimestamp() { return lamportTimestamp; }
    public void setLamportTimestamp(long lamportTimestamp) { this.lamportTimestamp = lamportTimestamp; }

    public int getSenderId() { return senderId; }
    public void setSenderId(int senderId) { this.senderId = senderId; }
}
```

6. Capa de Servicios (service/)

6.1. LamporkClock.java — Reloj Lógico de Lamport (Parte B)

Implementa el reloj lógico de Lamport. Usa AtomicLong para seguridad en entornos multihilo.

Reglas implementadas:

- **Regla 1 — Evento local:** incrementar() suma 1 al reloj ante cualquier evento interno del nodo.
- **Regla 2 — Recepción de mensaje:** actualizar(recibido) aplica $\max(\text{local}, \text{recibido}) + 1$, garantizando que la recepción ocurre después del envío y preservando la causalidad.

Listing 5: LamporkClock.java

```
package com.logistica.registro_distribuido.service;

import java.util.concurrent.atomic.AtomicLong;

public class LamporkClock {
    private final AtomicLong reloj = new AtomicLong(0);
}
```

```

    public long incrementar() {
        return reloj.incrementAndGet();
    }

    public long actualizar(long recibido) {
        long actual = reloj.get();
        long nuevo = Math.max(actual, recibido) + 1;
        reloj.set(nuevo);
        return nuevo;
    }

    public long getValor() {
        return reloj.get();
    }
}

```

6.2. AuthService.java — Autenticación por Token (Parte E)

Verifica que cada solicitud del cliente incluya el token válido en el header `Authorization`. Rechaza con **HTTP 401** si es nulo, vacío o incorrecto.

Listing 6: AuthService.java

```

package com.logistica.registro_distribuido.service;

import org.springframework.stereotype.Service;

@Service
public class AuthService {

    private static final String TOKEN_VALIDO = "token-secreto-123";

    public boolean validarToken(String token) {
        if (token == null || token.isBlank()) {
            return false;
        }
        return token.equals("Bearer " + TOKEN_VALIDO);
    }
}

```

6.3. NodoComunicacionService.java — Comunicación TCP (Parte A)

Centraliza el envío de mensajes entre nodos con sockets TCP. Implementa **framing por longitud previa**: envía el tamaño del mensaje como entero de 4 bytes antes del contenido, evitando solapamiento o truncamiento.

Listing 7: NodoComunicacionService.java

```

package com.logistica.registro_distribuido.service;

import org.springframework.stereotype.Service;
import java.io.DataOutputStream;
import java.net.Socket;

@Service
public class NodoComunicacionService {

    public void enviarMensaje(String host, int puerto, String mensaje) {
        try (Socket socket = new Socket(host, puerto)) {
            DataOutputStream out = new DataOutputStream(socket.getOutputStream());
            byte[] data = mensaje.getBytes();
            out.writeInt(data.length);
            out.write(data);
        } catch (Exception e) {
            System.err.println("Error enviando mensaje a " + host + ":" + puerto);
        }
    }
}

```

```

    }
}

```

6.4. HeartbeatService.java — Tolerancia a Fallos (Parte C)

Envía latidos cada 2 segundos a los otros nodos. Si el envío falla por excepción, detecta la caída y dispara automáticamente el algoritmo Bully.

Listing 8: HeartbeatService.java

```

package com.logistica.registro_distribuido.service;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class HeartbeatService {

    private final NodoComunicacionService comunicacionService;
    private final BullyElectionService bullyElectionService;
    private final List<Integer> otrosNodos = List.of(8082, 8083);
    public HeartbeatService(NodoComunicacionService comunicacionService,
        BullyElectionService bullyElectionService) {
        this.comunicacionService = comunicacionService;
        this.bullyElectionService = bullyElectionService;
    }

    @Scheduled(fixedRate = 2000)
    public void enviarLatidos() {
        for (int puerto : otrosNodos) {
            try {
                comunicacionService.enviarMensaje("localhost", puerto, "HEARTBEAT");
            } catch (Exception e) {
                System.out.println("Nodo en puerto " + puerto + " caído. Disparando
                    elecci n...");
                bullyElectionService.iniciarEleccion();
            }
        }
    }
}

```

6.5. BullyElectionService.java — Algoritmo Bully (Parte D)

Elección automática de coordinador. El nodo activo con mayor ID gana. Al detectar la caída del coordinador, se dispara sin intervención manual.

Listing 9: BullyElectionService.java

```

package com.logistica.registro_distribuido.service;

import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class BullyElectionService {

    private final NodoComunicacionService comunicacion;
    private final int myId = 1;
    private boolean enEleccion = false;
    public BullyElectionService(NodoComunicacionService comunicacion) {
        this.comunicacion = comunicacion;
    }
    public void iniciarEleccion() {
        if (enEleccion) return;
    }
}

```

```

        enEleccion = true;
        System.out.println("Nodo " + myId + ": Coordinador caído, iniciando elección...");
        boolean hayMasGrande = false;
        if (!hayMasGrande) {
            proclamarLider();
        }
    }
    public void proclamarLider() {
        System.out.println("Nodo " + myId + ": Soy el nuevo coordinador.");
        enEleccion = false;
    }
}

```

7. Capa de Controlador (controller/)

7.1. OperacionController.java — Endpoint REST (Partes A y E)

Expone el endpoint REST que recibe operaciones del cliente. Verifica el token antes de procesar. Responde HTTP 401 si es inválido, HTTP 200 si es correcto.

Listing 10: OperacionController.java

```

package com.logistica.registro_distribuido.controller;

import com.logistica.registro_distribuido.service.AuthService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/operaciones")
public class OperacionController {

    @Autowired
    private AuthService authService;

    @PostMapping
    public ResponseEntity<String> registrarOperacion(
        @RequestHeader("Authorization") String token,
        @RequestBody String operacion) {

        if (!authService.validarToken(token)) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                .body("Acceso denegado: Token inválido o ausente.");
        }
        return ResponseEntity.ok("Operación registrada correctamente.");
    }
}

```